TpvPlus Saldo

Monedero para eventos con pulsera NFC / RFID

Guia tecnica: servidor Ubuntu, app Android,
Odoo en Windows y app Windows para NFC

Odoo 20 | modulos feria_saldo y feria_saldo_pos | app TPVPlus

Equipo tecnico de Xtendoo - 28 de octubre de 2026

Indice

Indice 2

1. Arquitectura y decisiones 3

Componentes . . . 3

Decisiones de diseno . . . 3

2. Servidor Ubuntu (como lo montamos) 4

2.1 Stack . . . 4

2.2 Codigo y modulos . . . 4

2.3 Puesta en marcha . . . 4

2.4 Configuracion en Odoo . . . 4

2.5 API del kiosco . . . 5

2.6 Pruebas, copias y operacion . . . 5

2.7 Trampas de Odoo 20 que encontramos . . . 5

3. App Android TPVPlus 6

3.1 Como funciona . . . 6

3.2 Compilar e instalar . . . 6

3.3 Conectar el movil al servidor local . . . 6

3.4 Checklist de problemas . . . 6

4. Odoo en Windows 8

4.1 Opcion recomendada: Docker Desktop + WSL2 . . . 8

4.2 Alternativa nativa (sin Docker) . . . 8

4.3 Puesta a punto para el evento . . . 8

5. App Windows para escanear NFC (propuesta) 9

5.1 Fase 1 - sin desarrollo (recomendada para empezar) . . . 9

5.2 Fase 2 - app propia (si se necesita mas control) . . . 9

6. Estado, pendientes y comandos 11

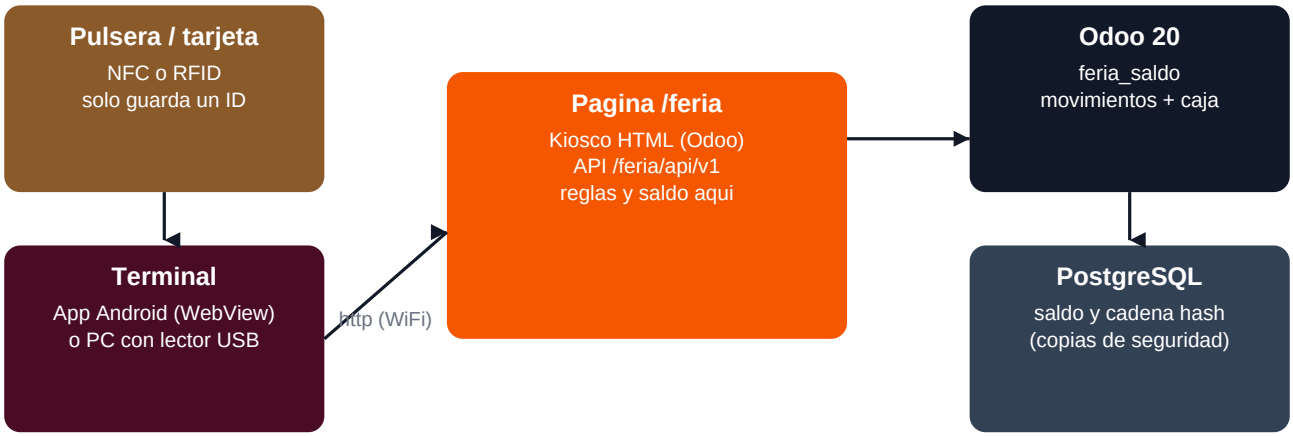
6.1 Estado . . . 11

6.2 Pendiente . . . 11

6.3 Chuleta de comandos . . . 11

1. Arquitectura y decisiones

El saldo vive **siempre en el servidor**. La pulsera o tarjeta NFC/RFID solo contiene un identificador (el UID del chip o un texto NDEF). Asi una pulsera clonada o perdida no pierde dinero: se bloquea el codigo en Odoo y se reasigna. Todas las reglas (saldo suficiente, idempotencia, caja) se aplican en el servidor; el terminal es solo una pantalla.



Componentes

Pieza	Que es	Donde esta
feria_saldo	Modulo Odoo 20: codigos, movimientos, caja, API y kiosco /feria	custom-tpv-plus (rama 20.0)
feria_saldo_pos	Puente opcional con el TPV de Odoo (cada cobro crea un pos.order pagado)	custom-tpv-plus
TPVPlus (Android)	WebView que abre el kiosco + lectura NFC nativa	custom-tpv-plus/TPVPlus
App Windows	Propuesta (apartado 5); no existe todavia	-

Decisiones de diseno

- **Kiosco sin login ni PIN**: tres botones (Cobrar, Recarga, Pedidos y saldo) y Caja abajo. Pregunta el cliente al entrar en cada flujo.
- **Movimientos solo de alta** (append-only) con uuid unico (idempotencia) y cadena de hash global; una anulacion es otro movimiento.
- **Concurrencia**: cada operacion usa su propia transaccion, bloqueo asesor de PostgreSQL y SELECT FOR UPDATE sobre el cliente; se confirma (commit) antes de responder.
- **Importes en Decimal / texto**, nunca float en la pagina.
- **Recarga en efectivo o tarjeta**: solo el efectivo cuenta para la caja; la tarjeta es informativa.
- **Sin autenticacion en la API**: el acceso se limita por red (Ajustes - Redes permitidas, lista CIDR). Pendiente HTTPS (apartado 6).
- **Enlace con TPV** opcional: el monedero nunca se bloquea si el POS falla; los pedidos pendientes se reintentan por cron cada 5 min.

2. Servidor Ubuntu (como lo montamos)

Entorno de pruebas en el portatil de desarrollo: Ubuntu, Docker y el stack **Doodba** de Xtendoo en `/home/xtendoo/Escritorio/odoo/odoo_20`.

2.1 Stack

Servicio	Funcion	Puerto local
odoo	Odoo 20 (imagen Doodba)	127.0.0.1:20069 (web), 20072 (longpolling), 20899 (debug)
db	PostgreSQL 18	interno
odoo_proxy / gatekeeper	Proxy hacia Odoo	-
pgweb	Visor web de la base	20081
smtp	Correo de pruebas	20025
wdb	Depurador	20984

`docker-compose.yml` es un enlace a `devel.yaml`. Los puertos se publican solo en 127.0.0.1, por eso el movil no llega directamente (ver 3.3).

2.2 Codigo y modulos

Los modulos estan en el repo **custom-tpv-plus** (rama **20.0**, creada copiando la 19). En `odoo/custom/src/addons.yaml` se declara el repo y Doodba enlaza los modulos en `odoo/auto/addons` al hacer `up`.

```
# addons.yaml (segundo documento)
ENV DEFAULT_REPO_PATTERN https://github.com/xtendoo-corporation/{}.git
custom-tpv-plus: ["*"]
```

2.3 Puesta en marcha

```
cd /home/xtendoo/Escritorio/odoo/odoo_20
docker compose up -d                      # arranca todo (regenera enlaces de addons)
docker compose exec odoo odoo -d feria_test -i feria_saldo --stop-after-init
docker compose restart odoo               # tras instalar/actualizar
# navegador: http://localhost:20069/feria  (kiosco)  y  /odoo (back office)
```

> **BASE DE DATOS UNICA**
>
> En `devel.yaml` la variable PGDATABASE vale **feria_test**. Si hay mas de una base, `/feria` responde 404 (Odoo no sabe cual usar). Si se cambia PGDATABASE, Doodba recrea una base vacia con ese nombre al reiniciar: dejar solo feria_test.

2.4 Configuracion en Odoo

- Usuario inicial **admin / admin**: cambiar la clave antes de abrir el servidor a nadie mas.
- Contactos: pestana **TpvPlus** del cliente - se le asigna el codigo de pulsera/tarjeta (UID en hexadecimal minusculas o texto NDEF). Ejemplo de prueba: el cliente *Estres* con codigo RFID de la base tpvplus de odoo_19.
- Ajustes - TpvPlus: **Redes permitidas** (CIDR, p. ej. 192.168.0.0/24), limites de importe, y **Vincular con el TPV** (config POS, metodo de pago; producto auxiliar Cobro monedero TpvPlus, ref TPVPLUS-COBRO).

- Menu TpvPlus: Resumen, Movimientos, Caja (contar / cerrar caja, dejar una cantidad o retirarla), Codigos.
- Usuarios: grupos **Consulta** y **Gestor** (privilegio TpvPlus). Ajuste y anulacion solo desde el back office.

2.5 API del kiosco

Metodo	Ruta (/feria/api/v1)	Uso
GET	/health /config	Estado y configuracion (moneda, redes)
GET	/contacts/lookup?code= | contact_id=	Identifica al cliente y su saldo
GET	/movements?contact_id=&limit= /movements/{uuid}	Historial; resolver operaciones pendientes
GET	/orders?contact_id=	Pedidos / cobros del cliente
POST	/movements	COBRAR o RECARGAR (uuid, importe; RECARGAR exige payment_method cash|card)
GET / POST	/cash	Estado de caja; POST {uuid, kind count|close, counted, kept, note}

Errores: `{error_code, message}` con CODE_UNKNOWN, CODE_BLOCKED, INSUFFICIENT_BALANCE, DUPLICATE, FORBIDDEN, LIMIT_EXCEEDED, VALIDATION_ERROR. Una operacion repetida con el mismo uuid devuelve el mismo resultado: el terminal puede reintentar sin riesgo de doble cobro. La operacion en vuelo se guarda en localStorage antes de enviar y se resuelve al volver la conexion.

2.6 Pruebas, copias y operacion

```
# tests (69) -- usar un puerto distinto del servidor en marcha
docker compose exec -T odoo odoo -d feria_test -u feria_saldo,feria_saldo_pos \
  --test-tags /feria_saldo,/feria_saldo_pos --stop-after-init --workers=0 \
  --max-cron-threads=0 --http-port=18079 --db-filter='^feria_test$'

# copia de seguridad (hecha en /home/xtendoo/Escritorio/odoo/backups_bd)
docker compose exec -T db pg_dump -U odoo -Fc feria_test > feria_test.dump
# ojo: una copia completa necesita tambien el filestore
```

En `feria_saldo/scripts` hay pruebas de estres (concurrencia) y e2e de navegador (kiosco, NFC, caja, offline) que se usaron sobre clones de la base.

2.7 Trampas de Odoo 20 que encontramos

- `ir.config_parameter.get_param` ya no esta: se usan getters tipados.
- `Environment.sudo()` no existe: `api.Environment(cr, uid, {}, su=True)`.
- Las rutas GET usan cursor de solo lectura: hay que declarar `readonly=False` si escriben.
- CSP estricta (script/style 'self'): nada de estilos inline en el kiosco.
- Los modulos `tpv_plus_pos_*` solo tienen la version subida; el front del POS (OWL 3) **no arranca** aun.

3. App Android TPVPlus

Proyecto en `custom-tpv-plus/TPVPlus` (Kotlin). Es un **WebView** que abre la direccion del servidor y anade lo que la web no puede hacer bien: lectura NFC nativa, pantalla siempre encendida y boton atras.

3.1 Como funciona

- **Primer arranque**: pide la direccion del servidor. `ServerConfig.normalize` anade `http://` si falta y `/feria` al final. Basta escribir `192.168.0.113:8080`.
- **Reconfigurar**: 5 toques rapidos arriba a la derecha.
- **NFC**: al acercar una pulsera lee el UID (hex minusculas) o el texto NDEF y llama a `__nfcTriggerRead` / `handleNfcScan` en la pagina. Solo se entrega a la pagina del servidor configurado.
- **Atras**: la pagina gestiona la vuelta con `tpvHandleBack` (no cierra la app).
- Trafico http en claro permitido (`usesCleartextTraffic`) porque el servidor de pruebas no tiene TLS.
- Icono adaptativo + monocromo con el logo TPV+.

3.2 Compilar e instalar

```
cd custom-tpv-plus/TPVPlus
./gradlew --offline assembleDebug testDebugUnitTest
# APK generado (copia): /home/xtendoo/Escritorio/odoo/TPVPlus-feria-debug.apk
adb install -r TPVPlus-feria-debug.apk      # o copiar el APK al movil y abrirlo
```

El APK es de **debug**: sirve para pruebas. Para repartir a terminales hay que generar uno **release firmado** (pendiente).

3.3 Conectar el movil al servidor local

Odoo solo escucha en 127.0.0.1:20069 del portatil. Para que el movil (misma WiFi) llegue se abre un puente con **socat** hacia la IP del portatil:

```
IP=$(ip -4 -o addr show wlp4s0 | awk '{print $4}' | cut -d/ -f1)
echo $IP
socat TCP-LISTEN:8080,bind=$IP,fork,reuseaddr TCP:127.0.0.1:20069
# en la app: $IP:8080      (p. ej. 192.168.0.113:8080)
# probar antes en el navegador del movil: http://$IP:8080/feria
```

> **LA IP CAMBIA**
>
> La IP del portatil cambia segun la red (192.168.0.113, 192.168.5.21...). Hay que repetir el comando con la IP actual y reconfigurar la direccion en la app. Si no carga: WiFi de invitados con aislamiento de clientes, cortafuegos, o la IP ya no es la del comando.

Riesgos de socat: deja Odoo abierto a toda la red local, sin cifrar (http) y sin registro de accesos. Usarlo solo en una red de confianza y mientras se prueba; ademas, activar **Redes permitidas** en Ajustes. Alternativa: cambiar en devel.yaml los puertos `127.0.0.1:` por `0.0.0.0:`.

Web NFC del navegador exige HTTPS, por eso en el movil se usa la app nativa y no Chrome.

3.4 Checklist de problemas

Sintoma	Causa probable
La pagina no carga	socat parado, IP cambiada, o el movil esta en otra red
404 en /feria	Hay mas de una base de datos; dejar solo feria_test

Sintoma	Causa probable
Pantalla antigua (login)	Cache del navegador; la app ya usa el kiosco nuevo
Codigo desconocido	El UID no esta asignado a ningun contacto (pestana TpvPlus)
Banner sin conexion	Perdida de WiFi; la operacion en curso se resuelve sola al volver

4. Odoo en Windows

> **NO PROBADO**
>
> Todo este apartado es una **recomendacion**: hasta ahora solo se ha ejecutado en Ubuntu. Hay que validarlo en un equipo Windows real antes del evento.

4.1 Opcion recomendada: Docker Desktop + WSL2

Reutiliza exactamente el mismo stack Doodba, asi que el comportamiento es identico al de Ubuntu.

Paso	Detalle
1	Windows 11 Pro/Enterprise (o 10 22H2), virtualizacion activada en BIOS, 16 GB de RAM recomendados
2	`wsl --install` (Ubuntu) y Docker Desktop con backend WSL2
3	Clonar el proyecto **dentro del sistema de ficheros de WSL** (`~/odoo_20`), no en C:\ (mucho mas rapido)
4	`docker compose up -d` e instalar el modulo igual que en 2.3
5	Quitar el bloqueo a 127.0.0.1: publicar el puerto 20069 en 0.0.0.0 (o usar el puerto del proxy) para que lo vean los terminales
6	Regla de Firewall de Windows: entrada TCP al puerto elegido solo para la subred local (perfil Privado)

4.2 Alternativa nativa (sin Docker)

- Python 3.12, PostgreSQL 16+ y Odoo 20 desde codigo fuente en un entorno virtual; `odoo-bin -c odoo.conf` como servicio (NSSM).
- Mas trabajo con dependencias (wkhtmltopdf, libs de Python) y ruta distinta a la de produccion en Linux: solo si Docker no es posible.

4.3 Puesta a punto para el evento

- **IP fija** (reserva DHCP en el router) para el servidor: evita reconfigurar las apps.
- **Energia**: desactivar suspension e hibernacion; portatil enchufado y con SAI.
- **Arranque automatico**: Docker Desktop al iniciar sesion, y reinicio automatico de los contenedores (`restart: unless-stopped`).
- **Copias**: tarea programada con `pg_dump` + filestore cada pocas horas a otro disco/USB.
- **Red del evento**: router propio, sin aislamiento de clientes, sin acceso desde Internet.
- **HTTPS** (pendiente): proxy con certificado interno; despues, fijar el certificado en la app (pinning).
- Cambiar claves de admin y de PostgreSQL; activar Redes permitidas.

5. App Windows para escanear NFC (propuesta)

Objetivo: que un puesto con PC use el mismo kiosco /feria que el movil, leyendo pulseras con un lector USB. Se mantiene el **mismo contrato** que Android: la app entrega el codigo a la pagina llamando a `window.handleNfcScan(uid)`. La logica y el saldo siguen en Odoo.

5.1 Fase 1 - sin desarrollo (recomendada para empezar)

- Un lector USB que funcione como **teclado (HID wedge)** escribe el UID y pulsa Enter. El kiosco ya lo captura (buffer de teclas) sin tocar nada.
- Abrir Edge o Chrome en modo kiosco: `msedge --kiosk http://IP:8080/feria --edge-kiosk-type=fullscreen`.
- Configurar el lector para emitir el UID en el mismo formato que usan las pulseras ya asignadas (hex minusculas, sin separadores) o normalizar en el servidor (`normalize_code`).

5.2 Fase 2 - app propia (si se necesita mas control)

Una carcasa fina que muestra el kiosco y lee el lector por **PC/SC**, el servicio de tarjetas de Windows.

Aspecto	Propuesta
Tecnologia	.NET 8 (WPF o WinForms) + **WebView2** + libreria PCSC-sharp. Alternativas: Flutter Desktop o Electron con nfc-pcsc
Lectores	ACR122U / ACR1252U u otros compatibles PC/SC (el UID se pide con el APDU `FF CA 00 00 00`)
Contrato con la pagina	Al detectar tarjeta: `webView.ExecuteScriptAsync("window.handleNfcScan('04a1b2c3')");` mismo formato (hex minusculas) que Android
Configuracion	Direccion del servidor en un fichero/ajustes de usuario; gesto oculto para cambiarla (como los 5 toques)
Estados	Sin lector (aviso en pantalla), lector conectado, lectura correcta, tarjeta ilegible; reconexion automatica del lector
Kiosco	Pantalla completa, instancia unica, arranque con Windows, sin barra de direcciones ni menu contextual
Datos del WebView	Carpeta de datos **persistente**: guarda la operacion en vuelo (localStorage) para resolverla tras un corte
Seguridad	Cargar solo el host configurado; secretos futuros con DPAPI; http solo en red interna hasta tener TLS
Distribucion	Instalador MSI/MSIX firmado; actualizaciones manuales al principio
Pruebas	Unitarias del parser de UID y del normalizador de direcciones; prueba manual con lector real; simulador que llama a handleNfcScan

```
// Esquema (C#)
var ctx = SCardContext.Establish();              // PCSC-sharp
monitor.CardInserted += (s, e) => {
    var uid = ReadUid(e.ReaderName);               // APDU FF CA 00 00 00
    webView.CoreWebView2.ExecuteScriptAsync(
        $"window.handleNfcScan('{uid.ToLowerInvariant()}')");
};
```

DECISION PENDIENTE

Empezar con la **fase 1** en el evento y construir la fase 2 solo si aparecen limites (lector que no emite como teclado, necesidad de feedback sonoro o de bloquear el equipo).

6. Estado, pendientes y comandos

6.1 Estado

Elemento	Estado
Modulo feria_saldo (kiosco, API, caja, back office)	Hecho y probado (69 tests, estres, e2e)
Puente feria_saldo_pos con el TPV	Hecho y probado
App Android (config de IP, NFC, icono)	Hecho; APK de debug
Odoo en Windows / app Windows	Solo propuesta (este documento)

6.2 Pendiente

- Fase 6: Kubernetes (K3s, CloudNativePG, MetalLB) y copias automaticas.
- HTTPS en el servidor y certificate pinning en las apps.
- APK release firmado; app Windows nativa si se decide.
- Portar `tpv_plus_pos_*` a OWL 3 (el front del POS de Odoo 20 no arranca); `xtendoo_pos_receipt` no tiene rama 20.0.
- Modo offline limitado: bloqueado por ahora (no se permite cobrar sin servidor).

6.3 Chuleta de comandos

```
docker compose up -d | restart odoo | logs -f odoo | down
docker compose exec odoo odoo -d feria_test -u feria_saldo --stop-after-init
socat TCP-LISTEN:8080,bind=$IP,fork,reuseaddr TCP:127.0.0.1:20069
cd TPVPlus && ./gradlew --offline assembleDebug testDebugUnitTest
adb install -r TPVPlus-feria-debug.apk
git -C odoo/custom/src/custom-tpv-plus push origin 20.0
```